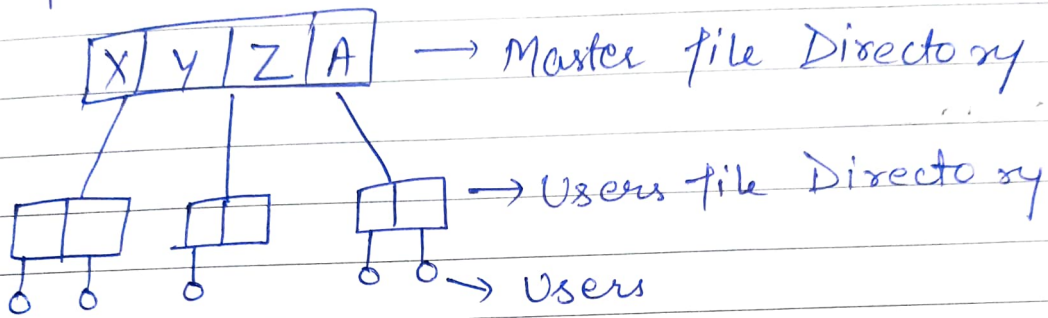


Overview of file Systems and DataBase Systems:-

file System → Basically a way of arranging files in a storage medium like a hard disk.

It organizes the file and helps in retrieval of files when they're required.



File Processing

DataBase System → A software that allows to create, update and retrieval of data in an organised way.
→ Centralised System

Database — It's a structured Collection of data that's stored in electronic device, this device can be text, media etc.



→ My Sql, Oracle, Snowflake, microsoft SQL server.
→ It's very to use.
(Create and manage file acca to our needs)
→ Less Cost
→ Direct Access.

file

DBMS

Structure

Acc. to definition

Acc. to defⁿ

Data Redundancy

More

Less

Backup & recovery

No

Yes

Query processing

No

Yes

Consistency of data

No

Yes

Complexity

Less

More

Security Constraints

Less

High

Cost

Less

High

Data Independence

Yes

No

User Access

Less

More

Sharing

less

More

Integrity Constraints

tough

easy

Data Abstraction

No

More

Characteristics of Data Base Approach
are these pts. //

Database users (Database administrator)

→ It interact with data- daily updating, reading and modifying given data.
Only the person responsible for updation of data.

1. Naive / Parametric End Users :-

Are those users who are unsophisticated who doesn't have DBMS knowledge, but they frequently use data in their daily life.

E.g. We are naive / Parametric (we use Railways apps)

2. System Analyst :-

Who analyse the requirement of Parametric end users, they check whether the requirement fulfill or not.

E.g. Check Application.

3. Sophisticated Users :-

These can be engineer, scientist, Business man who are familiar with DBMS knowledge, they can develop their own database applications acc. to their requirement.

They don't write the program code but they interact the database by writing SQL queries directly through Query processor.

4. Data Base Designers :-

Person or user who design the structure of database which includes tables, indexes, views,

He/she controls what data must be stored and how data items to be related.

It's responsibility of Database Designers to understand requirements of diff. user groups and then create a design which satisfies the need of all user groups.

5. Application Programmers :-

They're the back-end programmers who write codes for application programs.

→ They're simply Software Engineers.

6. Casual Users / Temporary users :-

who occasionally use/access database but each time when they access the database they require the new info,
for e.g. middle or higher level manager.

7. Specialized Users :-

→ Sophisticated users who write specialized database application that does not fit into traditional data-processing framework.

E.g. Computer-aided design systems, Knowledge base and expert system.

Importance of DBA :-

→ Manage and Control three level of Database

→ Internal

→ Conceptual

→ External level

→ Ensure and held responsible for Integrity and Security of data

[authorization]

- DBA also holds Countable that data is protected (Data loss) (Any chance of data loss kept at a minimum).
- provide Backups of data. (At regular intervals).

Roles & Duties of DBA :-

- Decide Hardware (Based on Cost, performance and efficiency of hardware)
Hardware that is an interface b/w end users and database.
- Manager data Integrity and Security (restricts data from unauthorized use)
(DBA eyes on relationships within data).
- Data Base Accessibility (permission to access data available)
(Who has right to change the content).
- DataBase Design (logical, physical and external model design).
- Data Base Implementation (Implements DBMS and checks database loading)
- Query Processing Performance (improving speed, performance and accuracy)
- Tuning Database performance (SQL commands → enhance performance of dB).
- Responsible for operation managements [restart or recovery process]
[Event/system failure]

DBMS Architecture :-

Database stores a lot of critical info to access data quickly and securely.

Hence, it's imp. to select correct architecture for efficient data management and system performance.

- 1-tier
- 2-tier
- 3-tier

1-tier Architecture

[User directly interact with the database]

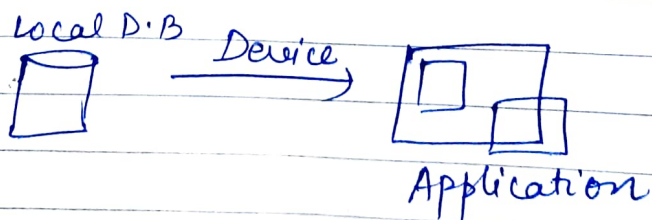
→ Setup is simple and often used in personal or standalone applications.

E.g.

→ A Microsoft Excel spreadsheet

- Everything : the user interface, application logic and data is handled on single system.
- User directly interacts with app., performs operations like calculations or data entry and stores data locally on same machine.

→ No external server or network connection is used.



→ Simple Architecture

→ Cost-effective

→ Easy to implement, (Easily deployed)
(mostly used in small projects)

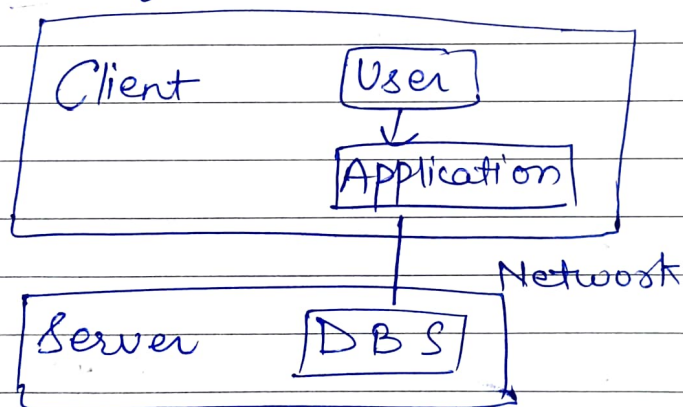
2-tier architecture

- Similar to client-server model.
- The application at client and directly communicates with database on server side.
- API's like ODBC and JDBC are used for this interaction.

Server side → Responsible for providing query processing and ~~tran~~ management functionalities.

Client side → the user interface and applications programs are run.

The application on client side establishes a connection with server side to communicate with DBMS.



The Client Layer sends a request to database layer which processes it and sends back the result.

This separation allows client to focus on user interface, while server handles data storage and retrieval.

- Easy to Access
- Scalable
- Low Cost (than 3-tier)
- Easy Deployment (than 3-tier)
- Simple

3-Tier Architecture

In this, there is another layer b/w the client and server.

The client does not communicate with server. Instead it interacts with an application server which further communicates with database system and then query processing and txⁿ management takes place.

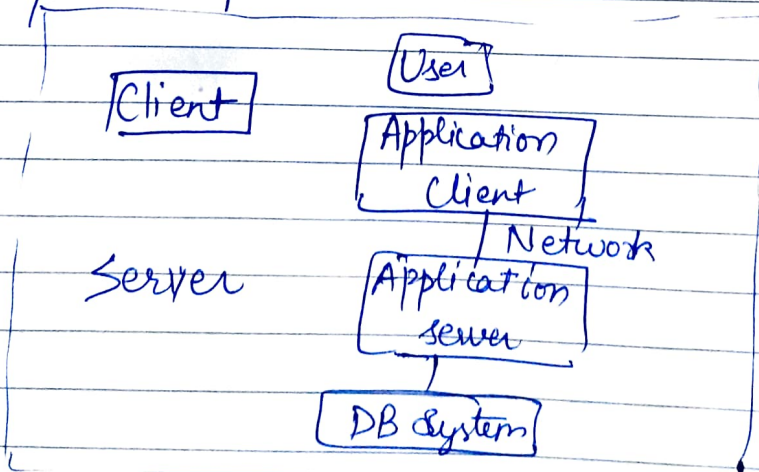
↳ Used in case of large web Applications.

for e.g. E-Commerce

Uses → You visit online store, search for product and add it to your cart.

→ Processing → System checks if product is in stock, calculates total price and applies any discounts.

Database → The product details, your cart and order history are stored in database for future reference.



Adv :-

↳ Enhanced Scalability

↳ Data Integrity

↳ Security.

Disadv :- More complex

Difficult to interact.

Database Languages in DBMS

↳ Specialized languages used to interact with a database.

They allow users to perform diff. tasks such as defining, controlling and manipulating the data.

- + DCL
- + DDL
- + DML
- + TCL

DDL → Data Definition Language.

↳ used to define database' internal structure and pattern of database.

↳ used to define and modify structure of database itself, including tables, Views

↳ It deals with creation and modification of db schema, but it doesn't deal with data itself.

CREATE : Used to create database objects like tables.

ALTER : Used to modify structure, such as adding a new column to table.

DROP : Used to delete database objects.

TRUNCATE : Used to remove all rows from table without affecting structure.

RENAME : Used to change the name of database object.

DCL → Data Control Language.

↳ used to control access permissions of users to database.

→ DCL commands are transactional, meaning they can be rolled back if necessary.

→ Grant : Gives user access to database

→ Revoke : Removes access or permissions from user.

Grant → ensuring that only authorized users can perform certain actions, such as selecting, inserting, updating or deleting data.

GRANT SELECT, INSERT ON Student TO User;

Revoke → remove previously granted permissions or privileges from users or roles.

REVOKE ALL PRIVILEGES ON students FROM User;

DML → Data Manipulation Language

↳ used to manage and manipulate data within a database.

→ we can perform various operations such as inserting, updating, selecting and deleting data.

SELECT : Retrieves data based on specific criteria.

INSERT : Add new rows of data

UPDATE : Modifying existing data

DELETE : Removes data from table

MERGE :- Merge 'upsert' operation.

CALL :- Executes stored procedure or f'n's.

LOCK TABLE : Prevents other users from accessing table while changes are being made.

TCL $\rightarrow$ Transaction Control Language

$\rightarrow$ Used to manage and control transactions in a database, grouping them into logical units.

$\rightarrow$ Ensures integrity and consistency of data during complex operations.

$\rightarrow$ Commit : Saves all changes made during current txⁿ to database.

Very useful in Banking sectors.

Database Operation
Commit

 — Command

Rollback : Used to restore database to its state at the last COMMIT, effectively undoing any changes made since that pt.

$\rightarrow$ Data consistency by allowing reversal of partial operations.

ROLLBACK;